UTM

UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING

SEMESTER 2 2025/2026

SECP3843 - SPECIAL TOPIC IN DATA ENGINEERING

SECTION 2

LECTURER: DR. ARYATI BINTI BAKRI

TUTORIAL 2 - APACHE SPARK

GROUP 17

STUDENT NAME	MATRIC NO
CHOH JING YI	A23CS0296
TAN ZHI MING	A23CS0189
LAU YAN KAI	A23CS0098

Table of Contents

1.0 Introduction

In the field of contemporary data engineering, one of the main challenges is how to manage large databases. As enterprises continue to accumulate more and more structured and semi-structured data, traditional processing methods are becoming more and more insufficient. They failed to meet the requirements, both in terms of processing speed and the ability to achieve system development without increasing complexity or cost accordingly.

Apache Spark solves this problem. It adopts memory processing and distributes computing tasks over multiple nodes. For heavy workloads that traditional systems are overloaded, Spark can handle them. Due to its compatibility with various data sources, including relational databases and original CSV files, it is very suitable for creating end-to-end ETL (extraction, conversion, loading) pipelines.

This report details how we operate according to the tutorial of "Creating a Simple ETL Pipeline with Apache Spark" written by João Pedro. The selected dataset is the Brazilian school census data compiled by the Brazilian Ministry of Education from the year of 2010 to 2021, which contains rich geographical and educational information.

This report covers the following overall workflows, importing raw datasets, using PySpark data framework to perform type security conversion, converting to Parquet column format, designing a star architecture data warehouse consisting of 12 dimensional tables and 1 fact table, loading data into the PostgreSQL database through JDBC, and using Power BI for visual demonstration. Throughout the process, the CRISP-DM method is used as a guiding framework.

2.0 Business Understanding

Every institution faces major challenges in effectively managing and interpreting more and more educational data. If the infrastructure is not prepared for large-scale applications, even simple analytical operations can become very time-consuming and labour-intensive when the data set contains tens of thousands of school records spanning years. By completing this tutorial, we learnt that:

1. Ingest and process the raw census dataset using distributed Spark computations on a Windows 11 local environment.

2. Fix Windows-specific compatibility issues including winutils.exe, HADOOP_HOME, and Spark temp directory permissions.
3. Convert cleaned data into Parquet format to improve query efficiency.
4. Design a Star Schema with 12 dimension tables and 1 fact table.
5. Load the processed tables into a PostgreSQL instance running inside Docker.
6. Verify schema and data as the database management interface.
7. Build an interactive Power BI dashboard showing enrolment trends by state and city.

3.0 Data Understanding

The open database of the Brazilian Ministry of Education provides relevant datasets. This dataset contains education census data for many years, from 2010 to 2021, and the data is stored separately in a separate CSV file every year.

The raw dataset includes information such as:

Item	Description
School locations	state, municipality, geographic region
School infrastructure indicators	water, electricity, sanitation, library, internet, computers
Student counts by education level and demographic group	the numbers of student enrolment
Teacher counts by education level	the numbers of teachers
Administrative classification	public/private, urban/rural

The datasets were originally stored in CSV format inside a compressed ZIP archive.

3.1 Dataset Structure

The original data set file (microdados_ed_basica_XXXX.csv) contains structured table education data, using semicolons as separators, and using Latin-1 (ISO-8859-1) encoding. The total size of uncompressed data in all years is about 2.2GB.

Key attributes used in this implementation:

Attribute	Type	Description
NO_UF	String	State name
SG_UF	String	State abbreviation (e.g. SP, RJ)

CO_UF	String	State code
NO_MUNICIPIO	String	Municipality name
CO_MUNICIPIO	String	Municipality code
TP_DEPENDENCIA	Integer	Administrative dependency (1–4)
TP_LOCALIZACAO	Integer	Location type (1=Urban, 2=Rural)
IN_AGUA_POTAVEL	Integer	Potable water indicator (0/1)
IN_INTERNET	Integer	Internet access indicator (0/1)
IN_BIBLIOTECA	Integer	Library indicator (0/1)
IN_COMPUTADOR	Integer	Computer indicator (0/1)
QT_MAT_BAS	Integer	Total basic education enrolments
QT_MAT_INF	Integer	Early childhood enrolments
QT_MAT_FUND	Integer	Primary education enrolments
QT_MAT_MED	Integer	Secondary education enrolments
QT_DOC_BAS	Integer	Total basic education teachers
NU_ANO_CENSO	Integer	Census year

Table 1 Key dataset attributes and their descriptions

3.2 Star Schema Design

This warehouse model is implemented in PostgreSQL using the Star Schema. The model consists of 12 dimensional tables and 1 central fact table. Unlike Snowflake schema, all dimensional tables are directly connected to the fact table without intermediate connection operations, so as to achieve faster analysis and query.

Dimension Tables Description

Table	Description
DIM_LOCAL	stores unique combinations of state and municipality (NO_UF, SG_UF, CO_UF, NO_MUNICIPIO, CO_MUNICIPIO)
DIM_TP_DEPENDENCIA	administrative dependency type
DIM_TP_LOCALIZACAO	urban or rural classification
DIM_IN_AGUA_POTAVEL	potable water availability
DIM_IN_ENERGIA_INEXISTENTE	no electricity indicator

DIM_IN_ESGOTO_INEXISTENTE	no sewage indicator
DIM_IN_BANHEIRO	bathroom availability
DIM_IN_BIBLIOTECA	library availability
DIM_IN_REFEITORIO	cafeteria availability
DIM_IN_COMPUTADOR	computer availability
DIM_IN_INTERNET	internet access
DIM_IN_EQUIP_NENHUM	no equipment indicator

Table 2 Dimension tables and their description

Fact Table Description

Table	Description
FACT_CENSO_ESCOLAR	stores 15 numeric metrics per school record, with foreign keys to all 12 dimension tables

Table 3 Fact table and its description

3.3 Schema Description

Fact Table: *FACT_CENSO_ESCOLAR*

Column	Type	Description
id	SERIAL	Primary key
QT_DOC_BAS	INTEGER	Number of basic education teachers
QT_DOC_INF	INTEGER	Number of early childhood teachers
QT_DOC_FUND	INTEGER	Number of primary education teachers
QT_DOC_MED	INTEGER	Number of secondary education teachers
QT_MAT_BAS	INTEGER	Total basic education enrolments
QT_MAT_INF	INTEGER	Early childhood enrolments
QT_MAT_FUND	INTEGER	Primary education enrolments
QT_MAT_MED	INTEGER	Secondary education enrolments
QT_MAT_BAS_BRANCA	INTEGER	Enrolments — White students
QT_MAT_BAS_PRETA	INTEGER	Enrolments — Black students
QT_MAT_BAS_PARDA	INTEGER	Enrolments — Mixed race students
QT_MAT_BAS_INDIGENA	INTEGER	Enrolments — Indigenous students
NU_ANO_CENSO	INTEGER	Census year

Column	Type	Description
ID_DIM_LOCAL	BIGINT	Foreign key to DIM_LOCAL
ID_DIM_TP_DEPENDENCIA	BIGINT	Foreign key to DIM_TP_DEPENDENCIA
ID_DIM_TP_LOCALIZACAO	BIGINT	Foreign key to DIM_TP_LOCALIZACAO
ID_DIM_IN_INTERNET	BIGINT	Foreign key to DIM_IN_INTERNET

Table 4 Fact table schema description

4.0 Data Preparation

In the process of data preparation, due to the data type compatibility problem between PySpark and PostgreSQL, as well as the specific Apache Spark problem of the Windows 11 version, multiple steps are needed to solve these problems.

4.1 Environment Setup on Windows 11

In the Windows 11 system, Apache Spark requires additional configuration, while in the Linux system, this configuration is not necessary. The following configurations has been successfully set up and installed:

Component	Version	Purpose
Python	3.11	Runtime for PySpark scripts
Java JDK (Adoptium Temurin)	11 LTS	Required by Apache Spark
Apache Spark	3.4.x	Distributed data processing engine
winutils.exe + hadoop.dll	Hadoop 3.3.5	Windows compatibility for Spark file operations
Docker Desktop	Latest	Runs PostgreSQL container
PySpark	pip install	Python interface to Spark
psycopg2-binary	pip install	PostgreSQL connection from Python
DBeaver Community	Latest	Database GUI for verification
Power BI Desktop	Latest	Dashboard visualisation

Table 5 Environment components and versions

The important Windows-specific environment variables configured:

```
HADOOP_HOME = C:\hadoop
JAVA_HOME   = C:\Program Files\Eclipse Adoptium\jdk-11.x
SPARK_HOME  = C:\spark
```

The Spark session was also configured with a local temp directory to avoid Windows permission errors with the default system Temp folder.

```
spark.local.dir = D:/Programming/Tutorial2_ApacheSpark/spark-temp
spark.driver.extraJavaOptions = -Djava.io.tmpdir=.../spark-temp
```

4.2 Data Extraction

The official data repository of the Brazilian government provides the original educational census data set. Subsequently, these data are extracted into the ..\Tutorial2_ApacheSpark\data directory in the project data directory. The total size of all CSV data sets in all years is about 2.2GB.

4.3 Reading CSV Data Using Spark

After setting inferSchema to true, Spark will scan the entire 2.2GB data to determine the type of column, which is a key problem that causes the reading process to run for more than an hour and not complete. This problem was solved by clearly handling the type conversion and setting inferSchema to false in the subsequent conversion steps. In this case, the entire IANA name iso-8859-1 must be used, because Spark does not allow the use of the coding alias latin1.

4.4 Data Cleaning

Data cleaning is not done through a separate cleaning script, but integrated as part of the Spark conversion process. The main pre-processing tasks include pattern reasoning, selection of relevant columns, deleting duplicate items during the construction of dimensional tables, and optimising data formats.

When creating a dimension table, we use Spark's dropDuplicates() function to delete duplicate records. This ensures that the records of each city and region only appear once in the corresponding dimension table. In order to help build the architecture of the data warehouse, we selected relevant columns from the original dataset, such as NO_REGIAO and NO_MUNICIPIO.

Since the selected attributes are mainly composed of classified geodomain information rather than continuous numerical indicators, there is no extensive outric value processing. In order to avoid unnecessary loss of valuable school-level records and the global deletion of missing values, such operations are not performed.

4.5 Conversion to Parquet Format

In order to improve query performance and storage efficiency, the cleaned data set is converted to Apache Parquet format. Compared with the original CSV file, the output file of Parquet is about five times smaller (420MB and 2.2GB). On a local Windows computer, the writing process takes about 5 minutes. Parquet format has the advantages of column-based storage, compression optimisation, faster analysis processing speed and smaller storage space.

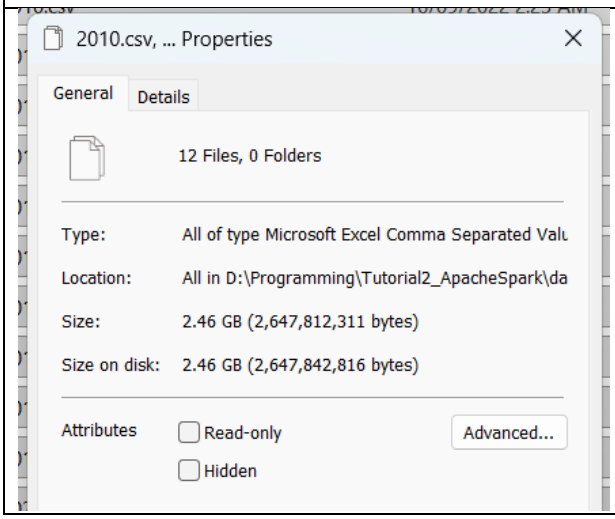
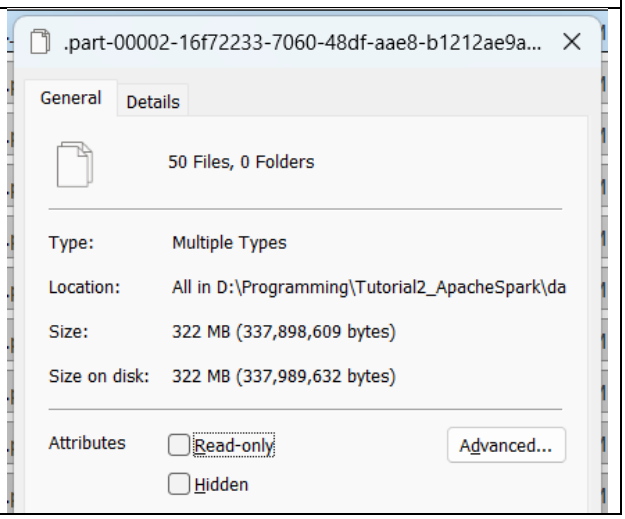
Original CSV file size	Converted Parquet file size
	

Table 6 Comparison of original file size and converted file size in Parquet

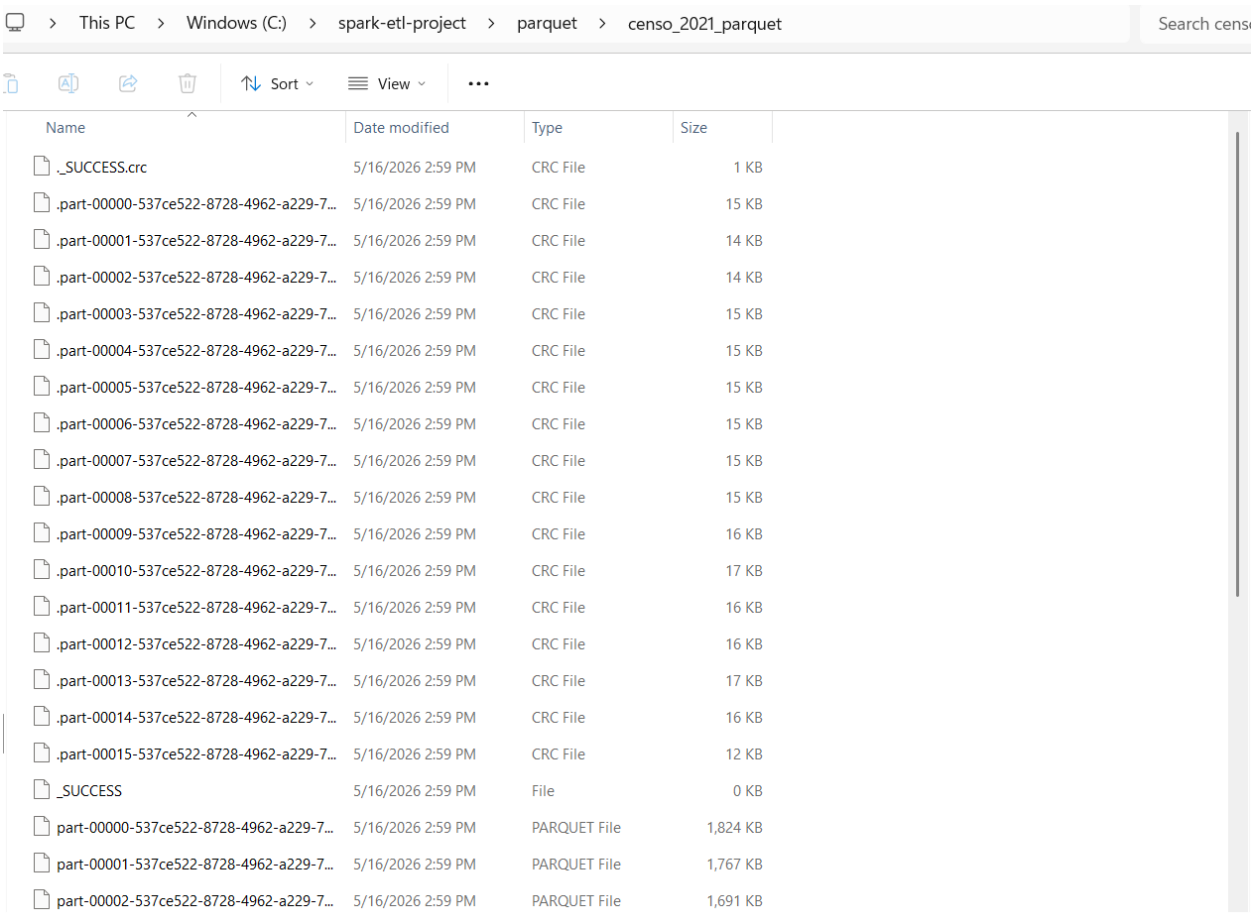
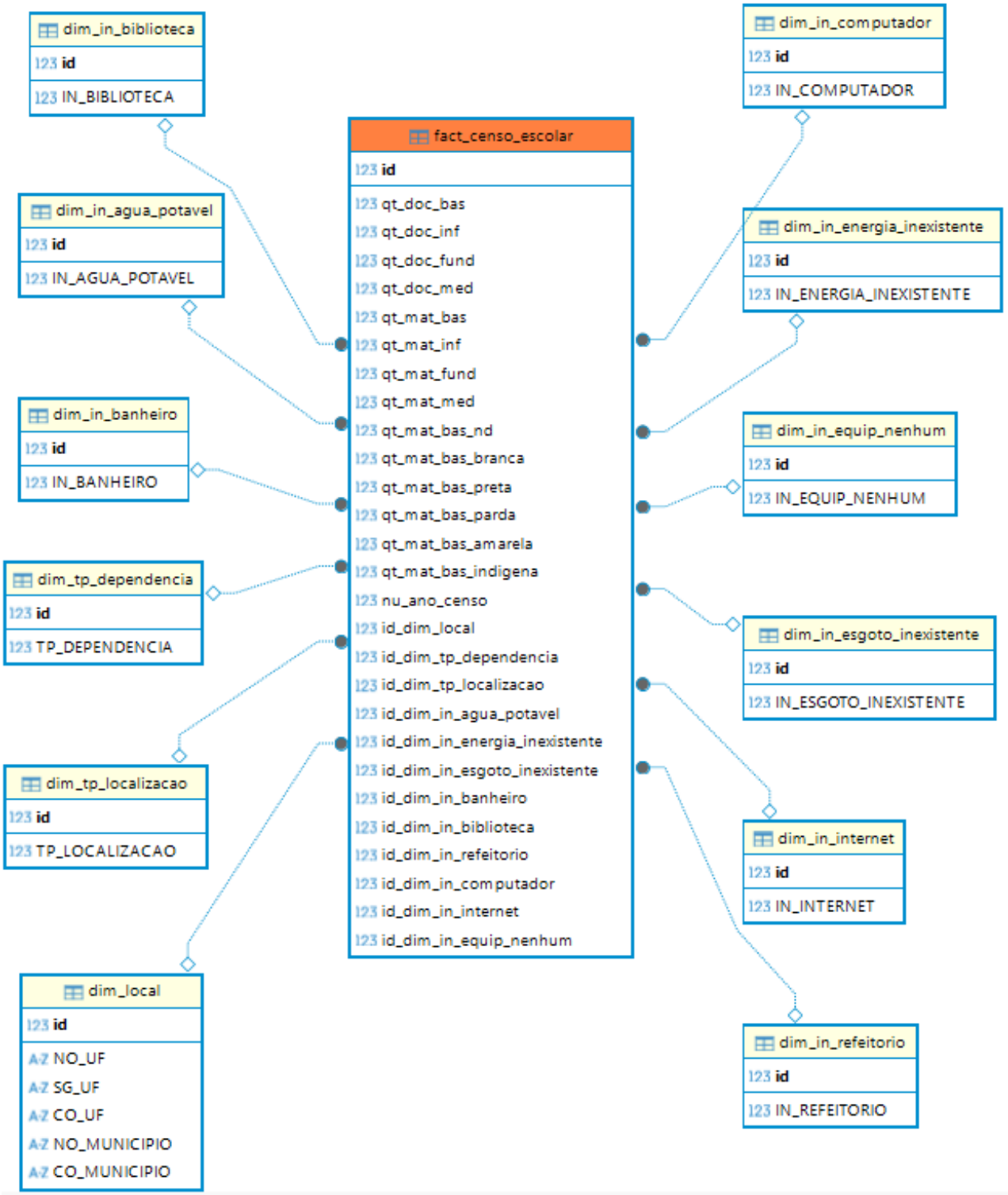


Figure 1 Generated Parquet files and _SUCCESS indicator after Spark transformation.

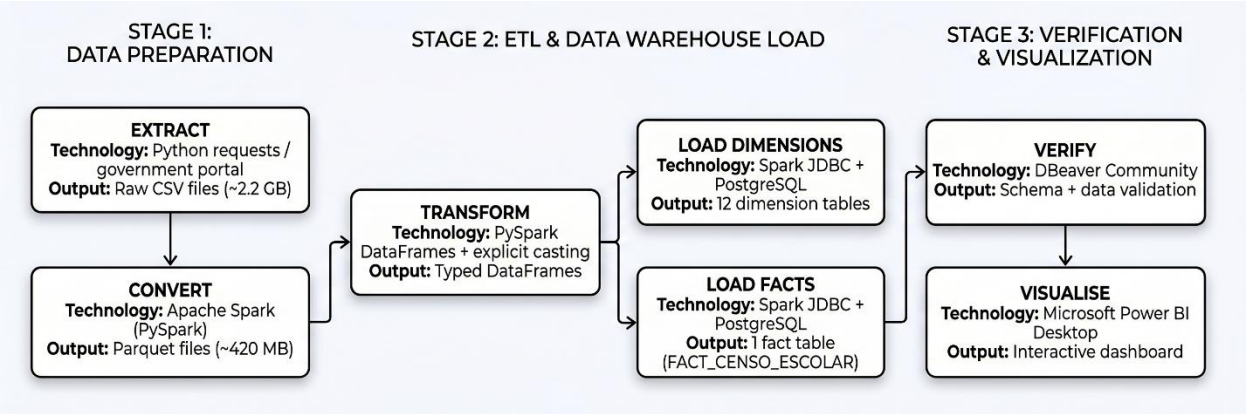
5.0 Modelling (Multidimensional Data Model)

The multi-dimensional warehouse model used in this tutorial is Star Schema. By avoiding the multi-hop connection found in the snowflake model, all 12-dimensional tables are directly connected to the main fact table, thus to achieving faster Power BI query performance.

5.1 Star Schema Design



5.2 ETL Architecture



Stage	Technology	Output
Extract	Python requests / government portal	Raw CSV files (~2.2 GB)
Convert	Apache Spark (PySpark)	Parquet files (~420 MB)
Transform	PySpark DataFrames + explicit casting	Typed DataFrames
Load Dimensions	Spark JDBC + PostgreSQL	12 dimension tables
Load Facts	Spark JDBC + PostgreSQL	1 fact table (FACT_CENSO_ESCOLAR)
Verify	DBeaver Community	Schema + data validation
Visualise	Microsoft Power BI Desktop	Interactive dashboard

5.3 PostgreSQL Warehouse Loading

PostgreSQL was run inside a Docker container with the following configuration.

```
docker run --name censo-postgres
    -e POSTGRES_USER=censo
    -e POSTGRES_PASSWORD=123
    -e POSTGRES_DB=censo_escolar
    -p 5432:5432 -d postgres:15
```

The JDBC driver JAR file of PostgreSQL is placed in the jars directory of Spark (C:\spark\jars\), which makes the connection between Spark and PostgreSQL possible. The result database contains 13 tables, 12 dimensional tables and the central fact table FACT_CENSO_ESCOLAR

6.0 Evaluation

The implemented ETL process was successfully executed on the local Windows 11 computer using Apache Spark and PostgreSQL. DBeaver is used to confirm that all 13 tables have been loaded correctly. For analysis and visualisation, the Power BI data dashboard is built based on the data warehouse.

6.1 System Functionality Evaluation

Component	Status	Notes
Environment Setup (Windows 11)	Successful	winutils.exe, HADOOP_HOME, JAVA_HOME
Docker PostgreSQL Container	Successful	All services started correctly on port 5432
Apache Spark Session	Successful	Local temp dir configured to avoid permission errors
CSV Read (inferSchema=false)	Successful	Read completed in seconds vs 1+ hour with inferSchema=true
Parquet Conversion	Successful	5x compression achieved (~420 MB output)
Dimension Table Loading	Successful	12 tables loaded with correct types via stringtype=unspecified
Fact Table Loading	Successful	FACT_CENSO_ESCOLAR loaded with 15 metrics and 12 foreign keys
DBeaver Verification	Successful	All 13 tables visible with correct schema
Power BI Dashboard	Successful	Enrolment by state and top 10 cities

Table 7 System functionality evaluation

6.3 Errors Encountered and Solutions

Error	Cause	Solution
inferSchema hang (1+ hour)	Spark scans all 2.2 GB to infer types	Set inferSchema=false, cast types explicitly
FileNotFoundException: data/*.csv	Script running from wrong directory	Use absolute path: D:/Programming/.../data/*.csv
latin1 charset error	Spark only accepts IANA encoding names	Changed to iso-8859-1
winutils.exe not found	Hadoop native library missing on Windows	Downloaded winutils.exe + hadoop.dll to C:\hadoop\bin

Error	Cause	Solution
NoSuchFileException in Temp	Spark cannot create dirs in Windows Temp	Set spark.local.dir to project folder
character varying vs integer	All columns string due to inferSchema=false	Added .option("stringtype", "unspecified") on JDBC write
PostgreSQL connection refused	Docker container stopped	docker start censo-postgres before running scripts

Table 8 Errors encountered and their resolutions

6.3 Limitations

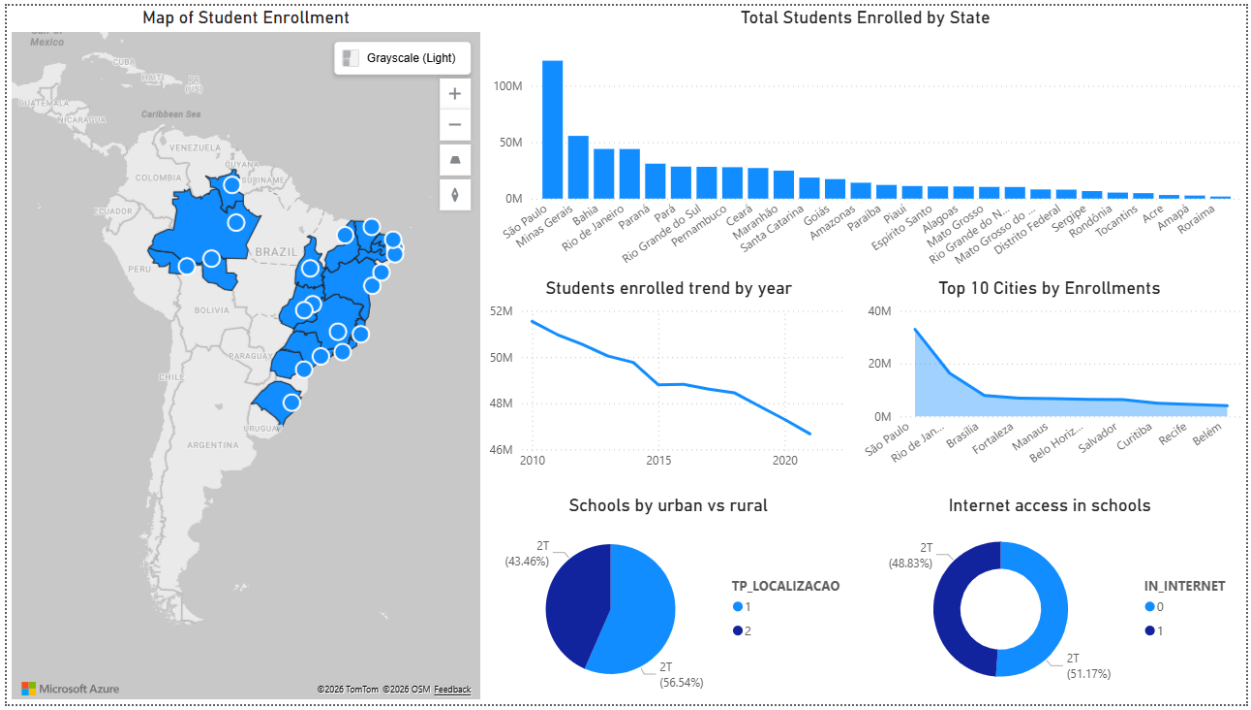
Some limitations have been determined. First of all, because no distributed cluster is used, all Spark tasks are executed locally on a Windows computer. Therefore, all the performance advantages provided by Spark's distributed architecture have not been realised.

Besides, on local hardware, it takes ten to thirty minutes to write JDBC operations containing millions of fact tables in multiple census years. In the actual production environment, batch loading tools such as COPY or pg_bulkload are much faster.

Also, the built-in Bing map engine needs to manually set the data category such as the state or province and the country column to accurately analyse the abbreviations of each state in Brazil. Although we use Power BI to fill in the map visualisation function of geographical visualisation, the shape map generated by the custom Brazilian TopoJSON file will provide more accurate and smoother results.

Deploying on hosting services such as multi-node Spark clusters or Databricks, using the date dimension for incremental data loading for multi-year trend analysis, using Spark structured flow processing to achieve real-time streaming processing functions, and using Apache Airflow to realise automated pipeline arrangement are all potential improvement directions in the future.

7.0 PowerBI



Visual	Type	Fields Used	Filter
Enrollments by State Map	Filled Map	dim_local[sg_uf], fact[qt_mat_bas] SUM	nu_ano_censo
Total Enrollments by State	Clustered Bar Chart	dim_local[no_uf], fact[qt_mat_bas] SUM	nu_ano_censo
Students Enrolled Trend by Year	Line Chart	nu_ano_censo, fact[qt_mat_bas] SUM	nu_ano_censo
Top 10 Cities by Enrollments	Clustered Bar Chart	dim_local[no_municipio], fact[qt_mat_bas] SUM	nu_ano_censo, Top N = 10
Schools by Urban vs Rural	Pie Chart	dim_tp_local[localizacao]	nu_ano_censo
Internet Access in School	Donut Chart	dim_in_internet[internet]	nu_ano_censo

Table 9 Power BI dashboard visuals

The Power BI data dashboard is directly based on the PostgreSQL Star Schema warehouse and consists of six interactive charts. All six visual interfaces realise cross-filtering which means that when a region, city or category is selected in any visual interface, all other interfaces will be automatically updated to reflect this selection so as to interactively explore the data warehouse from multiple analysis angles.

8.0 Reflection

Tan Zhi Ming (A23CS0189)
This project was a great opportunity to gain practical experience in creating a full ETL pipeline with Apache Spark and PostgreSQL. During the implementation process, some technical issues were faced, such as Docker configuration, PostgreSQL authentication, JDBC integration, and Spark environment setup on Windows. Addressing these challenges enhanced the ability to solve practical problems, and also gave a better understanding of real-world data engineering workflows. The project also enhanced the knowledge of distributed data processing, data warehouse modeling, schema design, and ETL pipeline development. In general, this tutorial contributed to the understanding of how modern big data technologies can be used to enable scalable analytical systems.

Choh Jing Yi (A23CS0296)
This Apache Spark ETL project was a very valuable learning experience for me. I faced several technical challenges at the beginning. First, I had to solve Python version mismatch errors between the Spark driver and worker by creating an Anaconda environment (spark_env). I also learned how to fix SSL certificate errors during the data extraction phase and how to properly connect the PostgreSQL JDBC driver to Apache Spark. Through this project, I gained experience in the complete Extract, Transform, and Load (ETL) process. I learned how to download raw data using Python, clean and reshape a massive dataset into a Snowflake schema using PySpark.The load the structured tables into a local PostgreSQL data warehouse. Overall, this project greatly improved my practical data engineering skills and my ability to troubleshoot real world software problems.

Lau Yan Kai (A23CS0098)
This tutorial is a meaningful project. New problems arise at every stage of the whole process. There was an error that the library is missing in the winutils.exe program. There is a permission problem in the temporary directory of Spark. When writing PostgreSQL, the character type didn’t match. Each problem needs to be investigated separately, analyse the error notification, and constantly try to solve it. The key is that all elements are interdependent. Any configuration error or lack of JDBC option in any Spark setting may quietly damage data or cause the whole process to crash. At the end of the project, I built a complete data warehouse from the original 2.2GB CSV data and established a set of troubleshooting methods to solve technical difficulties. I will apply these technologies in future data engineering tasks and projects.

6.0 Conclusion

This tutorial uses the Brazilian school census dataset to successfully build a complete Apache Spark data extraction, transform and loading (ETL) pipeline in the local Windows 11 environment to build an educational data warehouse. This implementation case demonstrates how Spark can effectively process large-scale structured datasets about 2.2GB of CSV data and load them into the PostgreSQL Star Schema data warehouse.

The technical challenges specific to the Windows system, including winutils.exe configuration, encoding name compatibility, type conversion through JDBC and Spark temporary directory permissions, have been identified, recorded and solved.

The final data warehouse contains 12 dimensional tables and 1 fact table (fact_censo_escolar), including indicator data on the enrolment rate, teachers and infrastructure of schools in Brazil in multiple census years. The Power BI report based on this warehouse provides interactive visual content, including state registration and analysis of the top 10 cities. The combination of tools such as Apache Spark, PostgreSQL, Docker, DBeaver and Power BI Desktop provides a practical and repeatable environment for modern data engineering workflows that can be expanded to larger data sets or cloud deployments.

7.0 Reference

I. Apache Spark Documentation. https://spark.apache.org/docs/latest/

II. Brazilian Ministry of Education Open Data Repository. https://download.inep.gov.br/

III. Docker Documentation. https://docs.docker.com/

IV. João Pedro. Creating a Simple ETL Pipeline With Apache Spark. Medium.

V. PostgreSQL Documentation. https://www.postgresql.org/docs/

VI. Microsoft Power BI Documentation. https://learn.microsoft.com/en-us/power-bi/

VII. Adoptium OpenJDK. https://adoptium.net/

VIII. DBeaver Community. https://dbeaver.io/